

C++ Multithreading & Concurrency Mini-Course

Phase 1: Basics of Threads

- ☐ Write a program using `std::thread` with functions
- ☐ Use `join()` to wait for threads
- ☐ Create threads using lambda functions
- ☐ Use `std::this_thread::sleep_for()` to simulate work

Phase 2: Thread Safety and Synchronization

- ☐ Use `std::mutex` to avoid race conditions
- ☐ Use `std::lock_guard` and `std::unique_lock`
- ☐ Demonstrate a race condition with and without a mutex
- ☐ Use `std::recursive_mutex` where needed

Phase 3: Advanced Thread Communication

- ☐ Use `std::condition_variable` for signaling
- ☐ Build a producer-consumer queue
- ☐ Use `std::atomic` for lock-free variables
- ☐ Explore `std::future`, `std::promise`, and `std::async`

Phase 4: Thread Pools & Patterns

- ☐ Create a simple thread pool from scratch
- ☐ Reuse threads for multiple tasks
- ☐ Queue incoming tasks and assign to idle threads

Phase 5: Debugging & Performance

- ☐ Detect and fix deadlocks
- ☐ Use Valgrind or ThreadSanitizer to detect race conditions
- ☐ Measure thread performance and scheduling delays

C++ Multithreading & Concurrency Mini-Course

Tools You'll Use

- [] `g++ -pthread`: Compile multithreaded code
- [] `std::thread`: Create threads
- [] `std::mutex`: Prevent race conditions
- [] `std::condition_variable`: Thread coordination
- [] `std::future`: Async task results

Optional (for deeper mastery)

- [] Use `std::shared_mutex` (C++17) for read/write locks
- [] Explore parallel algorithms (`std::for_each` with `std::execution::par`)
- [] Learn task-based concurrency with Intel TBB or OpenMP